

The Phase 1 Iterative Projection Is Not the Euclidean Projection

Measuring the incumbent `orthogonal_projection_iterative` against an independent exact reference — the evidence base for replacing it with an exact dual projection

July 2026

Status — measured (core finding); partial (replacement-algorithm probes)

The **core finding** — that the committed `orthogonal_projection_iterative` does not compute the true Euclidean projection — is **measured** and fully reproducible from `make_figures.py` (synthetic inputs, fixed seed; no `results/` run needed). It is the evidence base for the plan `docs/plans/PHASE1_DUAL_NEWTON_PROJECTION_PLAN.md`, which replaces the incumbent with the exact projection via the concave dual. The final §3.5 probes of the *replacement* algorithm (empty-cell stall, singular Jacobian, etc.) are **partial**: prototype measurements from an independent adversarial review that informed the plan, not properties of committed code.

Provenance

Date	2026-07-18 (verification); 2026-07-19 (independent adversarial re-confirmation)
Inputs	No results/ run. Synthetic candidates Y generated in <code>make_figures.py</code> with fixed seed 20260718. Lumped mass v built directly from <code>TorusMeshProvider</code> $T(R=1.0, r=0.6)$, $n_\theta=16$, $n_\phi=8 \Rightarrow V=128$, $\sum v = 22.7151$. Regimes: <i>interior</i> ($1/N + 0.03\mathcal{N}$), <i>binding</i> (near one-hot $+0.2\mathcal{N}$), <i>crisp</i> (one-hot $+0.02\mathcal{N}$, pre-clipped to $[10^{-8}, 1-10^{-8}]$, the PGD operating point).
Code under test	<code>orthogonal_projection_iterative</code> (<code>src/optimization/projection.py</code>), last modified 20637a2 (2026-06-26), <i>unchanged</i> on <code>feat/newton-projection</code> .
Reference truth	Dykstra's algorithm onto $\{\text{row}=1\} \cap \{\text{area}=d\} \cap \{\text{box}\}$ (Boyle–Dykstra; stop $\Delta \leq 10^{-14}$); cross-checked by a solver-free KKT stationarity residual, and by the 2026-07-19 review's three algorithmically-unrelated references (agree pairwise $\leq 4 \times 10^{-9}$).
Producing script	<code>make_figures.py</code> (beside this report; run from repo root)
Versions	Python 3.12.13, numpy 2.4.4, scipy 1.17.1, matplotlib 3.10.8
Anchors	KKT stationarity residual, true projection $\approx 10^{-15}$ vs incumbent 0.36 (crisp op-pt); $\max \text{incumbent} - \text{true} = 0.86$ (binding $N=10$), objective excess +64%; interior gap mechanism $ \text{incumbent} - P_{\text{aff}}(\text{rownorm } Y) = 5.95 \times 10^{-12}$

Abstract

Status: measured. Phase 1 relaxation projects the density onto the partition $(\sum_k u_{i,k}=1)$, equal-area $(\sum_i v_i u_{i,k}=d_k)$, and box $(0 \leq u \leq 1)$ constraints each PGD step, via `orthogonal_projection_iterative`

— documented and profiled as the *true Euclidean projection* and $\sim 86\text{--}91\%$ of Phase-1 wall time. It is *not* the Euclidean projection. Measured against an independent Dykstra reference, its fixed point differs from the true nearest feasible point by an input-dependent amount: ~ 0.009 when the box is inactive, rising to 0.31 ($N=4$) and 0.86 ($N=10$) in the crisp “binding” regime, and 0.19 at the realistic pre-clipped PGD operating point — with the incumbent’s squared distance to Y exceeding the true minimum by $+27\%$ to $+64\%$. A KKT stationarity test (which the true projection passes to 10^{-15}) puts the incumbent at 0.01–1.16. The mechanism is exact: the incumbent’s pre-loop *multiplicative* row renormalization makes it return the affine projection of the *row-normalized* input (residual 5.95×10^{-12}), not the projection of Y . Consequences (folded into the replacement plan): the correct validation target is the true QP, not equivalence to the incumbent; the replacement is a *correctness* upgrade (exact, idempotent, no $\sim 6.4 \times 10^{-8}$ clip-then-project noise floor), not a bit-for-bit drop-in.

1 Question

Each PGD iteration in Phase 1 replaces a post-gradient candidate $Y \in \mathbb{R}^{V \times N}$ (infeasible) with a feasible density by `orthogonal_projection_iterative` (V vertices, N cells). That routine is documented as computing the orthogonal (Euclidean) projection

$$\min_U \frac{1}{2} \|U - Y\|^2 \quad \text{s.t.} \quad \sum_k u_{i,k} = 1 \quad \forall i, \quad \sum_i v_i u_{i,k} = d_k \quad \forall k, \quad 0 \leq u \leq 1, \quad (1)$$

and the timing profile attributes $\sim 86\text{--}91\%$ of Phase-1 wall time to it. A planned replacement (`docs/plans/PHASE1_DUAL_NEWTON_PROJECTION_PLAN.md`) intends to compute (1) exactly and quickly, and to *validate* the replacement by equivalence to the incumbent. That validation strategy is only sound if the incumbent actually solves (1). **The question: does `orthogonal_projection_iterative` compute the Euclidean projection (1)?**

2 Method

We compare the incumbent’s output against an *independent* exact reference, on inputs that span the regimes Phase 1 actually visits.

- **Reference truth — Dykstra.** The exact Euclidean projection onto the intersection of the three convex sets $\{\text{row}=1\}$, $\{\text{area}=d\}$, $\{\text{box}\}$ is computed by Dykstra’s algorithm (each set’s Euclidean projection is closed-form; Boyle–Dykstra guarantees convergence to the exact projection of the intersection — unlike a naive alternating scheme). Stopped at successive-iterate $\Delta \leq 10^{-14}$; its own feasibility is machine- ε ($\text{row/area} \sim 10^{-13}$).
- **Solver-free certificate — KKT stationarity.** The exact projection must satisfy, on box-*inactive* entries, $(U - Y)[i, k] = \alpha_i + v_i \beta_k$ for some duals (α per vertex, β per cell). We fit (α, β) by least squares on the inactive entries and report the residual $\max |(U - Y) - (\alpha_i + v_i \beta_k)|$: ≈ 0 iff $U - Y$ lies in the projection’s dual span. (This stationarity residual is the clean discriminator; a *full* certificate that also fits the active-set sign conditions has a known false-negative weakness in the saturated regime — see §3.5.)
- **Regimes.** *Interior* ($Y = 1/N + 0.03\mathcal{N}$, box inactive), *binding* (near one-hot $+0.2\mathcal{N}$, box active on $\sim 45\text{--}70\%$ of entries), and the *crisp operating point* (one-hot $+0.02\mathcal{N}$, then clipped to $[10^{-8}, 1-10^{-8}]$ exactly as `pgd_optimizer.py` does before projecting). Lumped mass v is the real P_1 mass of $T(1.0, 0.6)$.

The incumbent is called exactly as PGD calls it (`tol=1e-8`); asking it for `tol=1e-12` makes it *raise* (it floors at $\sim 10^{-10}$ area error). *Anchor*: the true projection’s KKT residual reproduces to $\sim 10^{-15}$ and its crisp-op-pt feasibility to $\sim 10^{-13}$; the incumbent’s interior output reproduces $P_{\text{aff}}(\text{rownorm } Y)$ to 5.95×10^{-12} (§3.3).

3 Results

3.1 The incumbent’s fixed point is far from the true projection

Table 1 and Fig. 1 give the central result. Both the incumbent and the Dykstra reference are feasible (the incumbent to $\sim 10^{-9}$ row / 10^{-10} area, the reference to $\sim 10^{-13}$), but they are *different feasible points*: the gap $\max |\text{incumbent} - \text{true}|$ is 0.009 when the box is inactive and grows to 0.86 as the partition becomes crisp and N grows. Crucially the incumbent’s squared distance to Y is strictly *larger* than the true minimum (by +1.3% interior up to +64% binding): since (1) is strictly convex with a unique minimiser, a feasible point with larger objective is *not* the projection.

	interior $N=4$	binding $N=4$	binding $N=10$	crisp op-pt $N=10$
frac. entries clipped to 0 (true)	0.00	0.45	0.70	0.50
$\max \text{incumbent} - \text{true} $	0.009	0.314	0.858	0.189
objective excess $\frac{1}{2}\ U - Y\ ^2$	+1.3%	+27.4%	+64.2%	+30.9%
KKT residual (true / incumbent)	$10^{-16}/0.01$	$10^{-15}/0.63$	$10^{-15}/1.16$	$10^{-15}/0.36$

Table 1: The incumbent is not the projection. It is feasible but objective-suboptimal, and fails the KKT stationarity test the true projection passes to 10^{-15} . The gap grows with box activity (crispness) and N ; at the realistic pre-clipped operating point it is 0.19 (+31% objective).

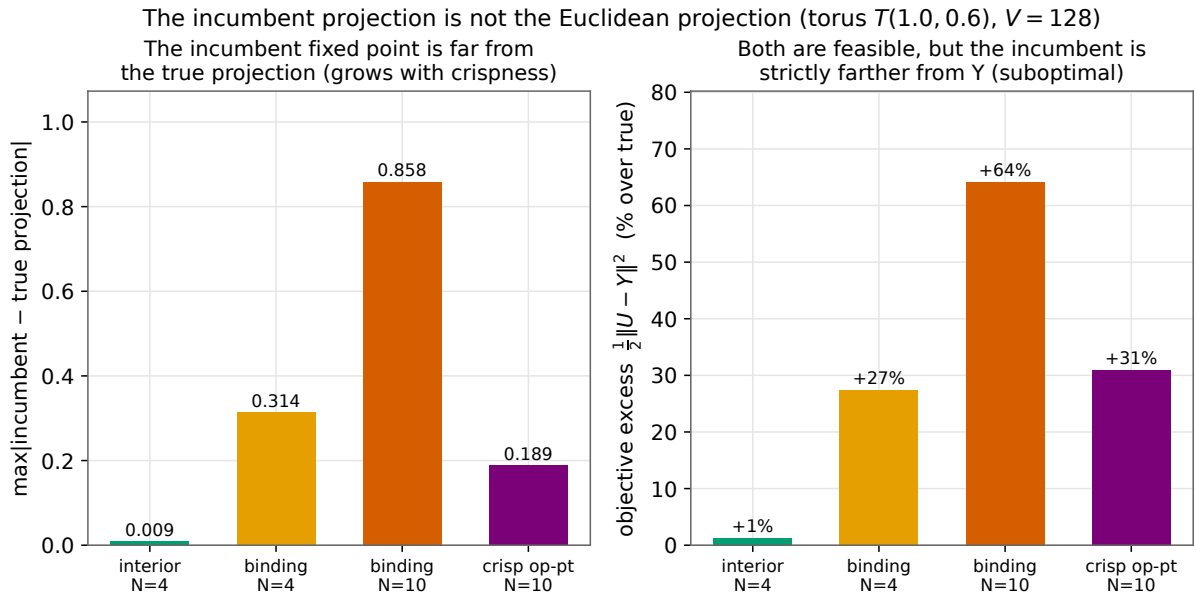


Figure 1: *Left*: $\max |\text{incumbent} - \text{true projection}|$ across regimes — small when the box is inactive (interior), large in the crisp/binding regime that is Phase 1’s actual operating point. *Right*: the incumbent’s objective $\frac{1}{2}\|U - Y\|^2$ excess over the true minimum — both points are feasible, but the incumbent is strictly farther from Y , so it is not the nearest feasible point.

3.2 A solver-free certificate: the incumbent fails the dual-span test

The true projection’s residual $U - Y$ lies exactly in the span of the constraint duals (KKT stationarity, Fig. 2 left): the Dykstra reference satisfies it to $\sim 10^{-15}$ in every regime. The incumbent’s residual is 0.01–1.16 — 13–15 orders of magnitude larger — so $U_{\text{incumbent}} - Y$ is

not a combination of the row and area dual directions, i.e. it is not the projection of Y . This certificate needs no external solver and is the cleanest single discriminator.

3.3 Mechanism: it is the multiplicative row renormalization

Why does the incumbent miss the projection even when the box is inactive? Its steps 1–9 are a *genuine* Euclidean affine projection onto $\{\text{row}=1, \text{area}=d\}$ (verified term-for-term against the C matrix at `projection.py:65–68`). But it first *multiplicatively* renormalizes each row ($\div$ row sum, `projection.py:54–57`), and its steps 10–12 (clip, then multiplicative row-renorm and column-rescale) are likewise not Euclidean projections. The net effect is exact (Fig. 2 right): the incumbent’s interior output equals the affine projection of the *row-normalized* input, $|U_{\text{incumbent}} - P_{\text{aff}}(\text{rownorm } Y)| = 5.95 \times 10^{-12}$, versus 7.0×10^{-3} against the affine projection of the raw Y . It solves a nearby but *different* problem.

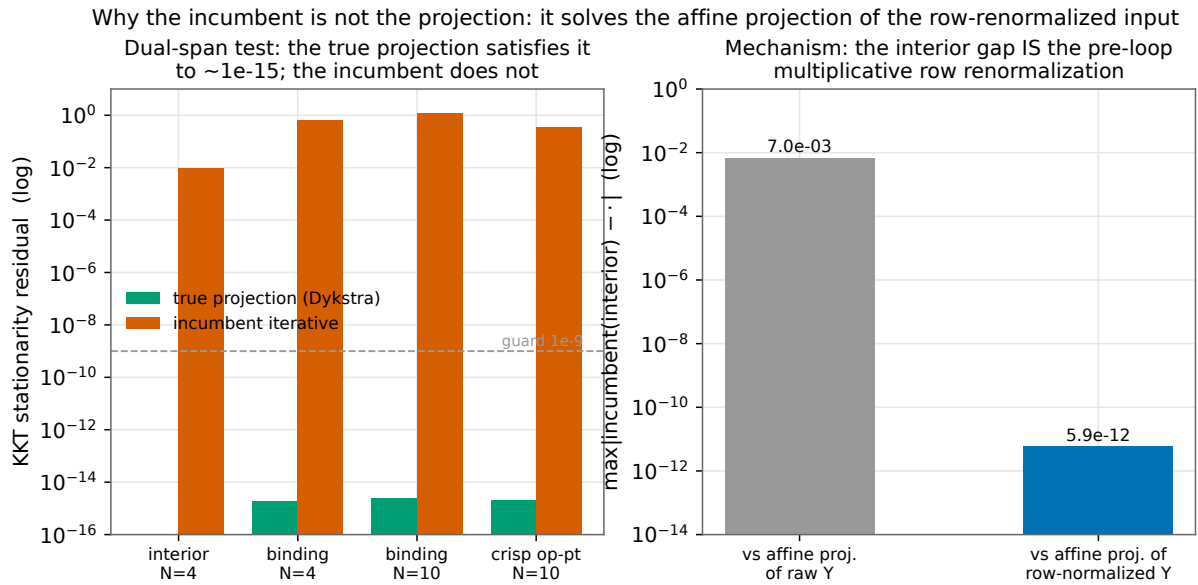


Figure 2: *Left*: KKT stationarity residual (log) — the true projection (green) passes to $\sim 10^{-15}$; the incumbent (vermillion) fails by 13–15 orders across every regime. *Right*: the interior gap is entirely the pre-loop multiplicative row renormalization — the incumbent matches the affine projection of the row-normalized input to 6×10^{-12} (blue), not the projection of the raw input (7×10^{-3} , grey).

3.4 Idempotency and the noise floor

Re-projecting an *exactly* feasible point barely moves it (3.5×10^{-11} ; any exactly feasible point is a fixed point of the map). But the PGD-level operation — clip to $[10^{-8}, 1-10^{-8}]$ *then* project, which the optimizer runs every step — moves an already-feasible point by 6.4×10^{-8} . That is the per-step noise floor that a true, idempotent projection eliminates (the pre-clip becomes unnecessary once the projection returns exact feasible entries, including hard 0s).

3.5 Implications, and an independent cross-check

The core measurement was **independently reproduced** on 2026-07-19 by an adversarial review using three algorithmically-unrelated exact references (closed-form affine projection, Dykstra, and dual L-BFGS with an exact simplex oracle), agreeing pairwise to $\leq 4 \times 10^{-9}$ and cross-checked

against `scipy trust-constr`; it found the gap reaches ~ 0.9 on harsher inputs, confirming and extending Table 1.

Two consequences for the replacement plan (`docs/plans/PHASE1_DUAL_NEWTON_PROJECTION_PLAN.md`):

1. **The validation target is the true QP, not the incumbent.** A correct exact projection *must* differ from `orthogonal_projection_iterative` by the amounts in Table 1; “match the incumbent to 10^{-8} ” is unachievable and wrong. Validate against Dykstra + the KKT conditions instead.
2. **The replacement is a correctness upgrade, not a drop-in.** It changes the per-step projection (hence the PGD trajectory): exact, idempotent, and free of the 6.4×10^{-8} noise floor. Keep the incumbent byte-identical as the default; validate the new path end-to-end for partition *validity*.

Partial (prototype) findings from the same review, on a not-yet-implemented dual solver, recorded here because they shaped the plan (they are measurements on prototypes, not committed code): (i) the full LS-fit KKT certificate — which also fits the active-set sign conditions — false-negatives (≈ 0.13) on the *true* projection when a row is fully saturated (one-hot, no inactive entry to pin α_i); the plan’s certificate must use the solver’s own duals. (ii) The area-constraint Jacobian of the dual is *structurally* singular ($J \cdot \mathbf{1} = 0$) and an empty cell makes an ε -ridge Newton step explode ($\sim 10^{10}$); the plan therefore adopts L-BFGS on the concave dual (robust to this) with a gauge-fixed Newton polish. (iii) The pre-clip is not a no-op: $\max |P(Y) - P(\text{clip } Y)|$ reaches 0.47 on realistic inputs, so it is removed on the new path. These are documented in the plan; this report’s *measured* claim is confined to §3.1–§3.4.

4 Conclusion

The Phase 1 constraint projection `orthogonal_projection_iterative`, documented and profiled as *the* Euclidean projection, does not compute it. It returns a feasible but objective-suboptimal, path-dependent point — exactly the affine projection of the multiplicatively row-normalized input — differing from the true nearest feasible point by up to 0.86 (+64% objective) in the crisp regime Phase 1 actually operates in, and failing the KKT stationarity test the true projection passes to 10^{-15} . It is also only weakly idempotent: the PGD clip-then-project step carries a $\sim 6 \times 10^{-8}$ per-iteration noise floor. This is the evidence base for `docs/plans/PHASE1_DUAL_NEWTON_PROJECTION_PLAN.md`: replacing the incumbent with an *exact* projection (via the concave dual) is a correctness upgrade as much as a speedup, and must be validated against the true QP — not against the incumbent it corrects.

Summary table

Quantity	Measured value	Source
Metric of (1)	Euclidean (not v -weighted)	<code>projection.py:106</code> ; §3.3
Max $ \text{incumbent} - \text{true} $ (crisp / binding)	0.19 / up to 0.86	Table 1, <code>make_figures.py</code>
Objective excess (binding $N=10$)	+64%	Table 1
KKT residual: true / incumbent	10^{-15} / 0.01–1.16	Fig. 2; <code>stationarity_residual</code>
Interior gap mechanism	$P_{\text{aff}}(\text{rownorm } Y)$, resid 6×10^{-12}	§3.3
Idempotency: bare / clip-then-project	3.5×10^{-11} / 6.4×10^{-8}	§3.4